

RESEARCH NOTE

Extending Agile Principles to Larger, Dynamic Software Projects: A Theoretical Assessment

Dinesh Batra, Florida International University, USA

Debra VanderMeer, Florida International University, USA

Kaushik Dutta, National University of Singapore, Singapore

ABSTRACT

The article evaluates the feasibility of extending agile principles to larger, dynamic, and possibly distributed software development projects by uncovering the theoretical basis for agile values and principles for achieving agility. The extant literature focuses mainly on one theory – complex adaptive systems – to support agile methods, although recent research indicates that the control theory and the adaptive structuration theory are also applicable. This article proposes that at least three other theories exist that are highly relevant: transaction cost economics, social exchange theory, and expectancy theory. By employing these theories, a rigorous analysis of the Agile Manifesto is conducted. Certain agile values and principles find theoretical support and can be applied to enhance agility dynamic projects regardless of size; some agile principles find no theoretical support while others find limited support. Based on the analysis and the ensuing discussion, the authors propose a framework with five dimensions of agility: process, design, people, outcomes, and adaptation.

Keywords: *Adaptive Structuration, Agile Manifesto, Agile Principles, Complex Adaptive Systems, Control Theory, Expectancy Theory, Social Exchange, Transaction Cost Economics*

INTRODUCTION

As business and technology environments change at an unprecedented rate, software development agility to respond to changing user requirements has become increasingly critical for software development performance (Lee & Xia, 2010). Software development agility is the

ability of an information system development (ISD) method to create change, or proactively, reactively, or inherently embrace change in a timely manner, through its internal components and relationships with its environment (Conboy, 2009). Agility is an organization's ability to sense and respond swiftly to technical changes and new business opportunities (Lyytinen & Rose, 2006). At its core, agility means to strip away as much of the heaviness, commonly associated with traditional software-development

DOI: 10.4018/jdm.2011100104

methodologies, as possible to promote quick response to changing environments, changes in user requirements, and accelerated project deadlines (Erickson, Lyytinen, & Siau, 2005). In response to the need for agility, lightweight agile software development methods have emerged as alternatives to process-heavy plan-based methodologies as organizations seek to deliver software more quickly (Abrahamsson, Conboy, & Wang, 2009), while simultaneously ensuring that the delivered software is of high quality and is closely aligned to the needs of the customer (Larman, 2003).

The call for such methods arose in 2001, with the publication of the Agile Manifesto (<http://agilemanifesto.org>), which has remained unchanged in a decade even as several agile methods have been proposed. The manifesto is based on four values: “individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan.” These values are accompanied by a set of twelve agile principles that provide guidance toward agile practice in development.

The manifesto was written by a group of practitioners interested in bringing together a number of lightweight methodologies, most of which now fall into the agile camp (Boehm & Turner, 2003; Qumer & Henderson-Sellers, 2008), including Scrum (Schwaber & Beedle, 2002), Extreme Programming or XP (Beck, 2000), Adaptive Software Development (Highsmith, 1999), and others. The agile movement grew out of practitioners’ impatience with heavier, plan-based methods, and their belief that there must be a better way. Indeed, the use of the word “manifesto,” a highly-charged word associated with revolutionary change, in the title was probably intentional – the authors wanted to highlight the radical differences between their agile methods and traditional plan-based approaches.

A decade later, the impact of the Agile Manifesto and its associated ideas is clear: agile methods have taken their place alongside more traditional approaches and are widely

used (McAvoy & Butler, 2009). Thousands of practitioners have signed their names in support of the Agile Manifesto (<http://agilemanifesto.org/sign/display.cgi>), while a 2008 survey by Dr. Dobb’s Digest suggests that up to 69% of responding organizations have adopted agile methods in some form, from pilot projects to full deployment of agile methods, and that respondents believe that their use of agile methods result in higher quality deliverables, more productive developers, and more satisfied stakeholders (Ambler, 2008).

However, there is evidence that supports the widely-held view that agile development has been applied only to small projects (Henderson-Sellers & Serour, 2005). Dyba and Dingsoyr (2008) present an extensive review of agile case study reports in the literature. Of the 33 projects referenced in this study, only four project teams had 20 or more members, and only one project team had a size greater than 23, at 60 members. Chow and Cao (2008) examined critical success factors in 109 agile projects. Of these projects, nearly 80% of project teams had fewer than 20 members. The Scrum methodology recommends projects teams of no more than six members (Schwaber, 2005). Beck (2005) states: “You probably couldn’t run an XP project with a hundred programmers. Not fifty. Not twenty, probably. Ten is definitely doable.”

It is widely accepted that agile development does, indeed, enhance agility although the supporting evidence is generally restricted to small projects (Boehm & Turner, 2003; Turk, France, & Rumpe, 2005). However, small projects account for only 17% of programming code (Beck & Boehm, 2003). Even free open-source software can be large and significant enough to disrupt commercial incumbents (Brydon & Vining, 2008).

Nevertheless, agile values and principles can provide a starting point to examine agility for large, dynamic projects. A few studies (e.g., Lindvall et al., 2004; Eckstein, 2004; Cao, Mohan, Xu, & Ramesh, 2009; Heeager & Nielsen, 2009; Batra, Xia, VanderMeer, & Dutta, 2010) have examined attempts to introduce agile methods in large development organizations.

Lindvall et al. (2002) defines a large software project as one requiring 20-40 or more developers. In this paper, we prefer the term “larger” project, which is meant as a project that does not have fewer than 20 developers.

Traditionally, larger and more distributed projects tend to follow more formal models and methods, such as those described in the Project Management Body of Knowledge (PMBOK) (Project Management Institute, 2004). These methods have their early roots in research in logistics and operations management theory (Kelley, 1961; Malcolm, Roseboom, Clark, & Fazar, 1959), and constitute a large and active area of research today. Although the discipline-oriented PMBOK practices have been popular, many experts (e.g., Boehm & Turner, 2003) question the effectiveness of the bureaucratic nature of discipline approaches in today’s dynamic business world. Many practitioners (e.g., Highsmith, 1999) have persuasively argued that excessive planning may run counter to the uncertainty-marked nature of the contemporary software project.

An important research issue, therefore, is to evaluate the feasibility of extending agile values and principles to large and potentially distributed information systems development projects to come up with a more balanced approach (Fernandez & Fernandez, 2008; Fitzgerald, Hartnett, & Conboy, 2006; Nord & Tomayko, 2006). Vinekar, Slinkman, and Nerur (2006) report that there is some empirical evidence that systems development organizations are attempting to employ both agile and organizational approaches. However, there are no guidelines that specify which agile principles are applicable for large projects. In fact, there is some uncertainty about the efficacy of agile methods. After a thorough evaluation of empirical studies in agile development, Dyba and Dingsoyr (2008) remarked (p. 852): “For agile software development, we believe the current state of theory and research on methods is clearly nascent.” One way to determine the applicability of agile values and principles is to examine each agile value and principle in the context of relevant theories, and determine the extent to

which each is supported by theory. We posit that principles with strong theoretical support will scale well as projects grow larger and, perhaps, become more distributed. Principles with weak or no theoretical support have probably not been challenged because smaller projects are likely to succeed simply because of lower complexity. After all, as Hilkka, Tuure, and Matti (2005) point out, excellent designers should be able to handle any small project.

Based on the supported principles, we propose a set of agility dimensions designed to serve as an *agility support framework* for larger, dynamic software projects where the traditional agility methods have not sufficed. Practitioners involved in large dynamic projects can select the agility dimensions most appropriate for their specific scenarios, balancing these with plan-based methods to enhance the agility of their projects. Researchers can extend this framework through empirical studies and case studies to develop a comprehensive framework for balancing agile and plan-based approaches.

The remainder of this paper is organized as follows. We first introduce key theories that are relevant to the discussion of the agile values. We then consider each of the twelve agile principles, searching for its theoretical support. For principles that are only partially supported, we suggest possible modified applications for larger, dynamic projects. We also point out principles that are largely not supported. Finally, we summarize the analysis and propose the agility support framework.

AGILE VALUES AND ORGANIZATIONAL THEORIES

The agile manifesto lists four essential values: individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. It is assumed that when a value is presented as A over B, that there is a significantly more emphasis for A than for B, and not just a subtle preference for A over B.

A literature search to unearth the theoretical basis of agile development revealed only one theory: *Complex Adaptive Systems* (CAS). The CAS theory (Highsmith, 1999; Vidgen & Wang, 2009) states that complex systems are like living organisms, and are made up of multiple interconnected elements and have the capacity to change and learn. These systems should not be viewed as dead, mechanistic, and linear machines. The CAS theory suggests that adaptation is significantly more important than optimization because complex systems cannot be predicted because of their emergent behavior (Meso & Jain, 2006). Emergence is a property of complex adaptive systems that creates some greater property of the whole, and can, thus, cause lack of predictability. Although the CAS theory supports some agile principles, and this discussion is covered in a later section, the theory cannot be used to explain the majority of the principles. For example, the CAS theory cannot explain why customers should be allowed to make changes late in the project (unless the customers compensate the vendors accordingly). The CAS theory does not address key indicators of the projects such as meeting budgetary goals, ensuring cost-benefit of the project, and completing the project in time.

Just as information systems development success is not a narrow dimension (Siau, Long, & Ling, 2010), information systems development agility is a wider notion than that proposed by the Agile Manifesto, which is employed only as an initial benchmark. As an example, a statement like the agile value “individual and interactions over processes and tools” can be interpreted by examining the agile principles that exemplify the value as well as by how the value is explicated in literature and practice. If the premise is that motivated developers can trump processes and tools, then the principle should be adapted based on theories that address motivation instead of assuming that only motivated individuals are allowed in the project. Similarly, the value “customer collaboration over contract negotiation” would necessitate examining theories related to trust between principal and agent

as well as on contracts. A software development method that does not address the issue pertaining to legal agreements on budget and time cannot work for projects that are large. By using agile values and principles as a starting point, examining the applicable theories, and adapting the values and principles, we can come up with a more relevant agility framework.

The Agile Manifesto covers both values and principles. Theories relevant to the agile values are examined first.

Individuals and Interactions over Processes and Tools

The underlying notion in this value is that motivated individuals can adapt processes and tools successfully and that teams should dynamically organize as needed. While evaluating agile practices, Turk, France, and Rumpe (2005) remark that motivation is one of the most important properties humans need in order to achieve ambitious goals with good quality results. There are several motivation theories, but the spirit of the Agile Manifesto is, perhaps, closely related to the *Expectancy Theory* (Vroom, 1964), which suggests that motivation can be increased by organizational rewards if efforts can lead to outcomes, outcomes can lead to the rewards, and the rewards are valued. One of the agile principles detailed later, states: “Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.” While the agile principle expects that the individuals are intrinsically motivated, which can be considered somewhat dubious, the Expectancy Theory is more specific in how to motivate individuals because it recognizes individual preferences on reward designs. Further, under the aegis of this theory, processes and tools can work alongside individuals and interactions.

Although motivation is important, equally important is control: should the control be formal or informal, and should it be guided by outcome, behavior, clan, or self-control? Maruping, Venkatesh, and Agarwal (2009) used Ouchi’s (1981) ideas and used the *Control*

theory to examine how agile methodology use interacted with requirement changes and the mode of control. A similar approach was used to determine control mechanisms in open-source software (Lin, Xu, & Xu, 2011). Other researchers have indirectly considered the issue of control (McAvoy & Butler, 2009).

Working Software over Comprehensive Documentation

The software project may be considered as a complex adaptive system (Highsmith, 1999), which contains complex interactions that lead to emergent results. Requirement changes may result simply because of a communication gap - what was documented by the analyst may be different from what was desired by the customer - or it may result because the complex interactions are better revealed when there is an actual product. Working software provides a concrete product to the customer, and the feedback can be used to intervene and make corrections early in the project. As explained earlier, the *complex adaptive system (CAS) theory* (Holland, 1992) can address the uncertain nature of dynamic software projects.

Customer Collaboration over Contract Negotiation

Two theories are related to this principle: one supports contract negotiation while the other supports customer collaboration in specific situations. The stakes are greater in a large project and the default scenario favors contract negotiation. The *Transaction Costs Economic theory* considers the transaction costs as the costs of contracting, and admits that the measurement of transactions costs poses formidable difficulties (Williamson, 1979). Two key determinants of transaction costs are bounded rationality (Simon, 1957) and opportunistic behavior. A contract is always incomplete because of the limitations of human mind (Williamson, 1979). However, a deliberately loose and incomplete contract, or no contract, will likely result in opportunistic behavior. The agile manifesto eschews the contract placing the customer at

an advantageous position, and the software vendor at a distinct disadvantage. The “customer collaboration over contract negotiation” value, thus, needs to be revisited especially for large, distributed projects where contracts cannot be avoided because the agreements underlying a contract actually protect both the software firm and the customer. A contract can cover some but not all contingent situations especially relating to the dynamic aspect of the project.

In long term relationships, however, the *Social Exchange Theory* (Blau, 1986), which states that social behavior can entail an exchange of goods, both material and non-material (e.g., symbols of approval or prestige), predicts that customer collaboration is possible. Thus, the goodwill and reputation engendered as both customer and vendor collaborate in addressing unforeseen situations can foster agility. Overall though, the agile value – customer collaboration over contract negotiation – is applicable only under narrow circumstances in large projects.

Responding to Change over Following a Plan

The *complex adaptive system (CAS) theory* (Holland, 1992), based on responding to emergent effects resulting from change, is an important underlying theory here. Another lens to examine this agile principle is the adaptive structuration theory (AST), which examines organizational change facilitated by different types of structures provided by advanced technologies, tasks, and organizational environments, as well as structures that actually emerge in social action (DeSanctis & Poole, 1994). A key concept of this theory is appropriation, which refers to the application of the structures in a particular context. Appropriation creates ‘structures in use’ when groups select and interpret specific structures from the ‘structural potential’ afforded to them by the artifact in consideration. The act of appropriation is a continual reflection, interpretation, and adjustment of structures in use.

In summary, the key theories that form the theoretical support for our framework, based on the agile values, are: Complex Adaptive System (CAS) theory, Expectancy theory, Control theory, Transaction Costs Economic theory, Social Exchange theory, and Adaptive Structuration theory. These theories are further examined by analyzing the agile principles, which are detailed and fine-grained explication of the agile values.

A THEORY-BASED ANALYSIS OF AGILE MANIFESTO PRINCIPLES

In the previous section, we identified key theories related to agile values. In this section, we assess each agile principle and evaluate if it can be supported by existing theories, and under what conditions. We summarize our analysis in Table 1. In many cases, theoretical guidelines indicate the need for modifying the principle to make it applicable to larger, dynamic projects.

1. *Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.* This principle is supported by the CAS theory. When a software project that has significant anticipated change is proposed, the customer may not be able to describe the requirements in concrete detail. Neither the user (i.e., customer), nor the analyst can predict emergent effects from interactions of parts of the system. Continual development and feedback can steer the project in the right direction (Baskerville & Pries-Heje, 2004) invariant of the project type – large or small, co-located or distributed. The availability of working in-progress software provides the customer with a tangible base from which to describe emergent requirements in greater detail. Problems in specifying exact user requirements, which are a key reason for system failure, can be mitigated by early and continuous delivery

of software, which can facilitate agility in larger, dynamic projects.

2. *Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.* This principle is largely not supported when applied to larger projects. Transaction Cost Economics theory suggests that applying such a principle can encourage opportunistic behavior on the part of the customer (Williamson, 1979). In large projects, major changes made late in software development can be expensive (Boehm, 1981; Nuseibeh & Easterbrook, 2000). To curb opportunistic behavior and account for cost escalations for late-stage changes, a large project is expected to involve a detailed contractual agreement, which in turn may necessitate process-heavy change-management methods that account for the repercussions of requirements changes late in the project timeline.

A contingent look at Transaction Cost Economics Theory (Williamson, 1979), however, reveals a possibility for agility, even in large and distributed projects. Normally, this theory considers transactions costs in the absence of trust between organizations. As an organization does repeated business with a customer or partner, trust and reputation ensue, which lowers transaction costs (Gulati, 1995). Trust can be examined through the Social Exchange Theory (Blau, 1986), which states that social behavior can entail an exchange of goods, both material and non-material (e.g., symbols of approval or prestige). Parties that give much to others generally expect to get much from them, and parties that get much from others are frequently under pressure to give much back to them. This process of influence tends toward equilibrium in the exchanges, and is one of the reasons that companies frequently have preferred relationships with their vendors.

In social exchange scenarios, a loss in a single project is not necessarily tantamount to

Table 1. Theoretical assessment of agile manifesto principles

Agile Manifesto Principle	Applicable Theory	Scalability to Large Projects
1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.	Complex Adaptive Systems	Emergence is a property of complex adaptive systems that creates some greater property of the whole. In large projects, continual delivery of working software facilitates assessment and feedback by the customer.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.	Transaction Cost Economics (works against the principle) Social Exchange	In large projects, Transaction Cost Economics theory predicts opportunistic behavior on the part of customer. However, Social Exchange theory can accommodate changing requirements, even late in development, if there is trust between the customer and the vendor.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.	Complex Adaptive Systems	In large projects, the build cycles for customer and for internal purposes need to have different time frames.
4. Business people and developers must work together daily throughout the project.	None	There is no theory that supports "daily" interactions between business people and developers.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.	Expectancy Theory (partial support)	Expectancy Theory can be used to define appropriate incentives to motivate individual members of the project, rather than building projects only around motivated individuals.
6. The most efficient and effective method of conveying information to, and within, a development team is face-to-face conversation.	None	Face-to-face conversation is not an efficient mechanism in large, distributed projects.
7. Working software is the primary measure of progress.	Complex Adaptive Systems	Similar to (1) and (3) above.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.	Expectancy Theory (partial support)	Quality of life is an important issue that needs to be considered as a motivation tool in all projects. However, this agile principle may conflict with agile principle (2).
9. Continuous attention to technical excellence and good design enhances agility.	None; the principle is too generic	Systems concepts such as decomposition, minimization of dependency between subsystems, and refactoring need to be considered.
10. Simplicity – the art of maximizing the amount of work not done – is essential.	Complex Adaptive Systems	Adaptation is significantly more important than optimization in large projects.
11. The best architectures, requirements, and designs emerge from self-organizing teams.	Control theory (partial support)	The agile literature does <i>not</i> indicate how self-organizing teams can form in large groups.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.	Adaptive structuration	Continual reflection and assessment is needed to reconfigure internal and external competencies to address rapidly changing environments.

an overall loss of profit for the vendor, given that customer and vendor satisfaction can lead to trust, goodwill, and dependency, and can increase switching costs. The key here is that repeated opportunistic behavior, especially on the part of the customer, needs to be prevented. When referring to relationships in contemporary times, Drucker (1997) notes: “these relationships have to be based on a common understanding of objectives, policies, and strategies; on teamwork; and on persuasion – or they do not work at all” (p. 2). Anecdotal evidence from industry supports the need for trust from both the client (Armour, 2007; Layman, Williams, Damian, & Bures, 2006; Ramesh, Cao, Mohan, & Xu, 2006) and the vendor (Groise & Mangin, 2007; Jain, 2006) perspectives.

If there is trust in the customer-vendor relationship, it might be possible to support this principle – “welcome changing requirements – even late in development” – in large, distributed projects. Conversations about changing requirements should be structured around “reprioritization” rather than “new, additional requirements”. However, both customers and vendors need to acknowledge that major change, especially late in development, involves costs across several dimensions: time, budget, functionality, and sustainability. A contract, therefore, is indispensable. Thus, transaction cost economics and social exchange provide theoretical lens to examine changes, especially late, in software development.

3. *Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.*

This principle is similar to the first agile principle, and is supported by the Complex Adaptive Systems theory. The developer needs a discrete time frame to deliver functionality, and the customer similarly needs a reasonable time interval for evaluation. In a large, distributed project with multiple integration points, however, preparing a fully-integrated and customer-friendly build (e.g., one that includes installers and other release-oriented features)

may itself require a week or more to prepare, and in a way, represents an opportunity cost that is much better spent on software development. On a two-week release cycle, too large a fraction of time is spent in releasing software rather than building increasing functionality. In such a cycle, the customer might see little change between releases, which may incur a large opportunity cost for a small benefit. This suggests that the timeframes cited in this agile principle might require slight relaxation in large and distributed projects. The actual timeframe will depend on how soon a customer can observe a discrete change so that feedback can be provided for the complex system to adapt.

Frequent build cycles have two main goals: (a) to ensure that the overall system remains consistently in a close-to-integrated state; and (b) to provide the customer with the means to evaluate overall progress and give feedback. To ensure that each customer-oriented release includes sufficient additional functionality to justify the work of the build, the build cycle can be separated into two parts: a frequent internal build cycle for each module, perhaps every two weeks, to ensure integration over time, and a slightly longer customer feedback cycle, perhaps every four weeks. Overall, it is evident that this agile principle, somewhat of a repeat of the first principle, has considerable theoretical appeal and can lead to agility in large projects.

4. *Business people and developers must work together daily throughout the project.*

There is no theory to support “daily” interaction of business people and developers. One way to examine this principle is from the notion of marginal value, which indicates that the marginal return decreases as more of a service is added (O’Connor & Faille, 2000). The amount of time a customer provides to the software developers may be highly useful in the beginning or at certain intervals of a project, but will result in diminishing returns if the interaction is on a daily basis, especially if the predominant activity in a project is coding.

There is a definite value in customer-developer interaction as dictated by the needs of a project, but working together on a daily basis may be overkill. Dyba and Dingsoyr (2008) note that the role of on-site customer seems to be unsustainable for long periods. Moreover in a distributed development team, where various development teams are working in different time zones, daily interaction with the customer may add quite a bit of overhead, in time, money, and reduced quality of life. This principle, therefore, cannot be applied in larger projects.

5. *Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.*

This principle has two statements – the first is somewhat vague, while the second has some theoretical support. One cannot embark on a software project hoping that only the most motivated individuals are available. In large and distributed projects, there will be individuals with both high and low intrinsic motivation levels. Thus, one cannot magically “build projects around motivated individuals.” Expectancy Theory (Vroom, 1964) suggests that motivation can be increased by organizational rewards if the rewards can be earned and are valued. The theory considers motivation resulting from expectancy (the probability that effort leads to performance), instrumentality (the probability that performance leads to work outcomes), and valence (the value attached by the individual to the work outcomes). Motivation can be increased by giving the developers “the environment and support they need” assuming there are adequate organizational resources. Suitable hiring procedures and training can ensure expectancy and instrumentality. To facilitate valence, the rewards have to be flexible enough to support individual preferences. One person may favor monetary awards, the second may favor recognition, the third may favor interesting work, the fourth may prefer travel, the fifth may abhor travel, and the sixth

may prefer flexible working time and working from home. A project manager has to be sensitive to such individual differences and, within the organizational constraints, customize the reward structure.

However, the mere fact that an individual is provided incentives does not guarantee performance. There is no theory that supports the hypothesis that rewarding employees alone will result in better performance. Accountability is a key tenet of performance evaluation and the issue of control is paramount; otherwise, an agile method can quickly disintegrate into chaos. The Control theory (Ouchi, 1985) is, therefore, relevant and is discussed in more detail under the agile principle 11 on self-organizing teams.

This agile principle, taken as a whole, has little theoretical support. The agile principle assumes that only motivated developers engage in software development. A better principle that indicates how work and rewards are linked and distributed, and control is established, is needed.

6. *The most efficient and effective method of conveying information to, and within, a development team is face-to-face conversation.*

This principle is effective in small, co-located group situations; however, it can be an inefficient method in several situations. There is no theoretical support for the principle. Schermerhorn, Osborn, and Hunt (2003) consider two aspects of communication—effectiveness, which pertains to accuracy, and efficiency, which pertains to its cost. Face-to-face conversation is clearly effective because of feedback and its clarifying effect, trust facilitation, and because of kinesics or nonverbal cues. In a large, distributed project, face-to-face conversation is not always an efficient mechanism, which can also be severely mitigated by temporal, geographic, and cultural distances (Ågerfalk & Fitzgerald, 2006). “Face-to-face” connotes physical proximity, which is sometimes difficult in large projects and mostly not feasible in distributed development scenarios. “Face-to-face” may be very useful at an early stage

to establish familiarity and trust; otherwise, there are a number of efficient communication mechanisms such as video conferencing, email, phone conferencing, discussion forums, text messaging, and phone calls that may be more efficient. Therefore, the “face-to-face” agile principle lacks theoretical support and needs a broader interpretation to make it applicable in larger, especially distributed projects, and should include personal and impersonal as well as synchronous and asynchronous methods of coordination and communication.

Using a common albeit diverse set of tools and practices for the entire project organization, including all multisourcing partners and customer stakeholders, reduces the need for synchronous communication. Communication in software development projects falls primarily into two categories: activity coordination (e.g., assigning tasks to developers, keeping track of bugs and test results) and decision-making (e.g., design discussions, requirements elicitation, and customer feedback). Technology can assist communication in both categories, providing the team with the support needed to get the job done (suggested by agile principle #5). A wide range of software is now available to support coordination, and to some extent, decision-making for projects in general and software development projects in particular. Anecdotal industry experience strongly supports the use of shared communication environments, in many cases (Jain, 2006; Layman et al., 2006; Ramesh et al., 2006), citing it as a significant positive influence on project success, and in one case, citing non-use (Korkala & Abrahamsson, 2007) as contributing to project failure.

7. *Working software is the primary measure of progress.* This principle is quite similar to the agile principles (1) and (3), and is supported by the CAS theory.
8. *Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.*

This principle usually translates to the XP-espoused forty-hour work-week rule to manage work-related stress and strain. This principle aims to protect the quality of life of developers and other participants. However, this agile principle is not consistent with some other agile principles. Transaction Cost Economics theory (Williamson, 1979) suggests that an over-emphasis on customer satisfaction, as suggested in agile principle #2, can result in a loose contract that could result in opportunistic behavior on the part of the customer. The desire to satisfy the client can disrupt sustainable development as developers rush to meet project deadlines that result in long and stressful days for developers (Batra et al., 2010). The software development industry suffers from a higher rate of burnout than other industries (Procaccino, Verner, Overmyer, & Darter, 2002). Client satisfaction and employee wellness do not seem consistent in this industry. Hence this principle may be meaningless given that the agile manifesto ascribes so much emphasis on the client especially on allowing changes late in development through principle #2.

Quality of life should be an important aspect when considering developer motivation, as is implicit in principle #5. Expectancy theory (Vroom, 1964) is again applicable here and needs to focus on work stress, which does not merely arise from task demands and long hours of working, but can also be caused by role ambiguities, role conflicts, ethical dilemmas, interpersonal problems, career development, and physical setting (Schermerhorn, Hunt, & Osborn, 2003). For example, in a distributed work environment, time differences can cause sleep deprivation since regular cross-continent meetings are likely to be held during the night time for one of the two parties. Overall, the agile principle is a reminder that today's project managers need to find ways to manage developer stress so that burnout is minimized and sustainable development becomes feasible; however, the principle needs to be harmonized with other principles.

9. *Continuous attention to technical excellence and good design enhances agility.*

This principle is almost axiomatically true – technical excellence and good design will make any project better, regardless of size or level of distribution. However, the principle is rather generic and not suitable for theoretical evaluation; it should be made more specific. Good design is important, but so are many other facets such as good requirements, good coding, and good communication.

The more important issue is: how do you achieve good design in a large project? The intended purpose of the agile proponents is that refactoring should be done continually to ensure that the resulting code can be reused and easily maintained. Refactoring refers to the structuring of software to remove duplication, improve communication, simplify, or add flexibility without changing its behavior. Refactoring may be possible in large projects if two prerequisite activities are completed before refactoring is actualized: the project is decomposed into smaller parts, and dependencies are minimized between subsystems.

Projects suitable for agile development generally have one or few parts; however, a large software project is automatically complex because invariably it has many interacting parts (Reeves, 1999). A project viewed as a system can be decomposed into subsystems, which may be recursively divided into smaller subsystems, where the subsystems are less complex because they contain fewer components and relationships. Each module exposes a clear and unchanging interface through which other modules can access its functionality. When integrated, the full set of modules work together to provide the overall functionality of the system. Although the decomposition principle is well-known, it is rarely discussed in agile development references. This may be so because the agile projects are usually small in size.

Principles of good design dictate that low coupling, i.e., the degree of dependency between modules, and high cohesion, i.e., the degree of focus in a module's responsibility, are desirable

characteristics (Stevens, Myers, & Constantine, 1999). Low coupling is of particular interest, since this minimizes dependencies among modules. These dependencies are realized in the module interface implementations (Brehner, 2005). Minimizing dependencies between modules at the time of interface design enables module encapsulation (Blaha & Rumbaugh, 2005), which makes a large distributed project more amenable to agility principles. Maruping, Venkatesh, and Agarwal (2009) report a project that had 151 teams working on an enterprise-wide project; each team was responsible for developing a single module. Kahkonen (2004) suggests various methods that teams can use to communicate and coordinate with each other.

Many organizations have recently restructured their operations, replacing isolated departments with teams of people that are expected to be more flexible and creative in accomplishing work tasks (Eisenberg, Goodall, & Trethewey, 2006). A number of small, agile teams, potentially working at different locations, can work semi-independently on separate subsystems (Beavers, 2007). Modular design allows an organization to scale agile principles. For example, Scrum can be managed as a hierarchy of Scrum of Scrums (Schwaber, 2005). Each agile team can then actively engage in continual refactoring. In fact, developers should anticipate changes and enhancements, and incorporate design agility through parameterization.

10. *Simplicity – the art of maximizing the amount of work not done – is essential.*

This principle is supported by the CAS theory and can be scaled to large, distributed projects. The notion of simplicity is borrowed from the Lean Software Development (Poppendieck & Poppendieck, 2003) practices and represents a deliberate effort at identifying and reducing wasteful work. Achieving optimization necessitates clear and fixed specification of parameters such as user requirements and quality; however, in dynamic situations, cost and time become more important, and the need for adaptation requires that extraneous work

be minimized. Simplicity allows adjusting the scope and even quality if time and cost are the more important attributes of the project. Thus, this principle has theoretical support, and should scale very well to large, distributed projects.

11. *The best architectures, requirements, and designs emerge from self-organizing teams.*

This agile principle has limited theoretical support. Self-organization is an underlying precept in CAS for enabling adaptation. As the environment changes, organisms evolve and restructure to adapt to address the changing conditions. However, it is not clear how self-organization can be facilitated in large projects. The issue is one of control – how much control should be attributed to the project manager, to the team, and to the individual developer. Outcome, behavior, clan, and self-control are mechanisms to ensure that developers achieve the objectives of a project. Maruping, Venkatesh, and Agarwal (2009) used Ouchi's (1981) ideas and used the Control theory to examine how agile methodology use interacted with requirement changes and the mode of control. McAvoy and Butler (2009) found that the high level of empowerment of a cohesive software development team undertaking an agile project may be one of these negative factors, as empowered, cohesive teams can exhibit problems such as groupthink or the Abilene Paradox.

A self-organizing team is not always the best structure (Schermerhorn, Osborn, & Hunt, 2003, p. 207), because the characteristics of any self-managing team – high involvement, participation, and empowerment – must be consistent with the values and culture of the organization. Eisenberg and Goodall (2004) state that there is such a thing as too much autonomy, and that most employees do not want complete control over and accountability for their day-to-day decision making. Further, if the group size is large, it is plausible that some group members may attempt to get a free-ride by coasting while the work is done by others (Jensen & Meckling, 1976).

Jacques (1996) argues that hierarchy is necessary to ensure accountability, and that group authority without group accountability is dysfunctional. He states that “managerial hierarchy will remain the only way to structure unified working systems with hundreds, thousands, or tens of thousands of employees” (p. 248). Each team in a large project, therefore, should have a manager or coordinator that is responsible for meeting the goals. Some degree of hierarchy is necessary even in a fairly decentralized project structure, especially for larger and more distributed projects.

Another aspect that determines the team structure is the set of cultural norms of the organization, which in turn, is affected by the cultural norms of the region and country where the organization is located. Thus, the offshore office of an American or a British company may not find the same success with self-managing team structures as the central office. Hofstede and Hofstede (2005) have identified five dimensions of national culture: power distance, uncertainty avoidance, individualism-collectivism, masculinity-femininity, and long-term/short-term orientation. An offshore office located in a country with high power distance (how rigidly divides between supervisor/supervisee roles are maintained) may find the self-organizing team notion novel, if not downright strange.

Hierarchy is inevitable today given that a large number of software companies in countries such as the US and the UK have Asian offshore offices that carry out the programming tasks. In the latter countries, hierarchy is the preferred structure although it seems that at least in software companies, hierarchy is not resulting in undue bureaucracy. While it is generally accepted that in today's software development environment the traditional authority-heavy role of the project manager needs to give away to director and coordinator role (Nerur, Mahapatra, & Mangalaraj, 2005), self-organization in large projects does not seem to be a viable structure in the near future.

Table 2. *Agility dimensions*

Agility Dimension	Theory Base	Justification
Process	Complex Adaptive Systems	Iterative test-driven process facilitates agility. Simple methods enhance agility.
Design	Complex Adaptive Systems	Decomposition, minimization of dependency between sub-systems, parameterization, and refactoring enhance agility.
People	Expectancy Theory Control Theory	Motivation mechanisms need to be flexible and somewhat customized to the individual. Controls should be based more on informal mechanisms backed up by formal rules.
Outcomes	Transaction Cost Economics Social Exchange Theory Complex Adaptive Systems	Trust enhances agility while contracts prevent opportunistic behavior. Within this framework, the decision-maker is able to juggle costs, time, scope, and quality. Simplicity enhances the ability to make such trade-offs.
Adaptation	Adaptive Structuration Theory	The ability to reflect and learn during a project enables agility methods to be adapted and appropriated.

12. *At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.*

More than one theory can support this principle. For example, if one takes an organizational learning perspective, then the Action Learning Theory, which focuses on double-loop learning (Argyris, 1982; Nerur & Balijepally, 2007), may be applicable. In double-loop learning, the entities involved (individuals, groups or organization) question the values, assumptions and policies that led to the actions in the first place.

A related but more appropriate lens to examine this agile principle is the adaptive structuration theory (AST), which examines organizational change facilitated by different types of structures provided by advanced technologies, tasks, and organizational environments, as well as structures that actually emerge in social action (DeSanctis & Poole, 1994). Cao et al. (2009) consider agile methods as artifacts that facilitate an adapted structure due to social action of the participants in the process. The artifact itself, the task that uses the artifact and the organizational environment can be potential sources of structure. In essence, AST highlights structuration, which refers to the act of bringing

rules, resources, and other structures into action, and appropriation, which refers to the application of the structure in a particular context. Appropriation creates ‘structures in use’ when groups select and interpret specific structures from the ‘structural potential’ afforded to them by the artifact in consideration. The act of appropriation is a continual reflection, interpretation, and adjustment of structures in use.

IMPLICATIONS FOR THEORY

Table 2 summarizes our evaluation of the theoretical support for the agile principles. A few patterns emerge from this analysis. Principles 4, 6, and 9 do not have theoretical support while principles 2, 5, 8, and 11 have limited theoretical support. Principles 1, 3, and 7, all based on the CAS theory, point to iterative development modes. Principles 10 and 12 are also theoretically supported: principle 10 underscores simplicity in analysis, design, and coding, and principle 12 advocates appropriation and adaptation of structures brought out by interaction of artifacts, tasks, and organizational environment.

The Agile Manifesto has four values and twelve principles. As may be evident from the

discussions in this paper, there is overlap between agile values and principles, and among principles. Five principles have full theoretical support, four have partial theoretical support, and the remaining three have no theoretical support. Underlying the manifesto, there are just three agility-enhancing theoretically-supported themes: iterative development, simplicity, and adaptation. Various agile methods have emerged, and while the practices employed in these methods usually follow the Agile Manifesto, it seems that even newer principles such as pair programming, which have no theoretical support, are added. The Agile Manifesto has remained unchanged since it was proposed about a decade ago. Perhaps, it is time to revisit the manifesto, and extend it so that its dimensions not only have theoretical support, but also provide specific guidelines for enhancing agility in projects of all sizes.

In the spirit of establishing more rigorous facets of agility, a few researchers have proposed agility dimensions. Meredith and Francis (2000) proposed four dimensions of organizational agility (but not software development in particular): agile strategy, agile processes, agile linkages, and agile people. For outsourcing environments, Sarker and Sarker (2009) proposed three dimensions with sub-dimensions in each dimension: resource agility (people, technology), process agility (methodology, environment awareness, temporal-bridge), and linkage agility (cultural mutuality, communicative relationship). For software development, we propose an *agility support framework* along five dimensions.

Process Agility: This dimension pertains to the iterative delivery method, which is supported by the CAS theory. The iterative delivery mechanism supports both the customer as well as the developer. Developing and testing software components frequently, internally, keep the overall system in a close-to-integrated state at all times. However, delivering integrated, customer-ready software at the same frequency may hamper project progress;

instead, a frequency approved by the customer may be more useful. Methods and processes that are simpler are more likely to foster agility.

Design Agility: This dimension pertains to incorporating agility in the software through design techniques such as decomposition, minimization of dependency between sub-systems, parameterization, and refactoring.

People Agility: This dimension pertains to motivating developers and other personnel as well as their teams, and is supported by the expectancy theory. The dimension also addresses quality of life issues of the personnel. The creative energies of developers need to be harnessed to direct them to the project goals; however, informal control mechanisms need to be established while the formal control mechanisms need to be used as back-up or in situations that resonate with the culture of the team.

Outcomes Agility: This dimension considers the customers and providers and addresses how the various outcomes such as cost, schedule, scope, and quality are balanced and adapted. Decision time is an important success factor in agile development (Misra, Kumar, & Kumar, 2009). Responsive trade-off decisions by an authorized body like a steering committee can enhance agility. Trade-offs should consider cost-benefit rather than a bureaucratic commitment to initial success indicators. Simplicity enhances the ability to make such trade-offs. To prevent opportunistic behavior, a contract can establish the framework and benchmark for these outcomes. Within this framework that protects both the customers and the providers, trust can provide a catalyst for fostering agility.

Adaptation Agility: There is empirical evidence that even agile methods need to undergo adaptation (Aydin, Harmsen, Slooten, & Stegwee, 2005). "Agility mindset" may be defined as the ability to reflect and learn during a project so that methods can be adapted and appropriated. Eventually, methods are guidelines that are placed in a

complex web of requirements, outcomes, constraints, social interactions, developer capabilities, tools, and culture. Adaptation can be incorporated in any project by setting aside milestones for “agility retreats” where the focus is on reflection, learning, and appropriation.

IMPLICATIONS FOR PRACTICE

The agile manifesto was proposed by practitioners; thus, it is inevitable that the agility dimensions proposed in the paper will have significant implications for practice. In fact, agile development issues have been largely discussed in the practitioner arena but the theoretical analysis has largely been missing. Although the original authors of the Agile Manifesto defined their values and principles in opposition to planning-oriented approaches, the results of our study here suggest that there may be no reason to adopt a “pure” agile or plan-oriented approach; rather, elements of both may be judiciously combined to meet the needs of a software development project. This proposition, even as it awaits definitive empirical support, is very significant for practitioners.

The agility dimensions provide a more comprehensive view of agility, which can be applied to larger, dynamic projects. For example, project managers and team leads always need to keep an eye on project management aspects such as the quality, the cost (and the incremental benefit with respect to an incremental cost), and the project duration. Agile development does not directly focus on these attributes but practitioners are accountable on these measures of success, which can be tricky because of the trade-offs. A strategic project, for instance, has cost and quality targets but these attributes may sometimes pale in comparison to being first in the market. In addition, a large project that needs to be completed quickly may require an assessment of people and process agility. The project manager, team leads, and stakeholders may ask: Do we have skills required to complete this project? How long will our developers be

able to work sixty-hour weeks? What might motivate our developers? Can we outsource a portion or phase of the project? What might be a practical time interval for iteration? Will we have a calcified mindset or one that is open to learning as the project proceeds? How can we ensure that the change control process is not an adversarial, legalistic process but involves a transparent prioritization process that recognizes all stakeholders and allows for mutually agreeable trade-offs in the development schedule?

Agile development principles deserve a lot of credit for invoking the kind of thinking that leads to agility. In a large, dynamic project, however, a systems and comprehensive approach is required to evaluate a number of dimensions and achieve trade-offs. The five factors proposed in this paper - process, design, people, outcomes, and adaptation – provide a useful template for practitioners to analyze the complex nature and requirements of contemporary software development projects.

CONCLUSION

We believe that this paper is unique in its attempt to assess and adapt the agile principles for agility for larger and dynamic projects by employing a multi-theory based approach. We have examined the values and principles defined in the Agile Manifesto for theoretical support, and have proposed an agility support framework that can facilitate agility in large, dynamic software projects. This framework suggests five agility dimensions: process, design, people, outcomes, and adaptation.

Future research can employ this framework to study software development agility. The immediate step is to verify the dimensions using empirical research. As an example, multiple case interviews can be conducted involving projects that involve more than twenty developers and have witnesses a fair amount of requirement changes. The questions can be generated based on the five agility dimensions. Further, open-ended questions can be asked on

critical success factors that enhanced agility in a project. The transcripts can be analyzed to assess if the five dimensions were critical in enhancing agility. Similar questionnaire-based studies may also reveal such factors. Another approach is to examine case studies to identify factors considered in adapting a development method to facilitate agility.

Overall, our analysis of the Agile Manifesto reveals that it does not have strong theoretical support. Although about seven out of ten organizations have adopted agile methods, it is likely that these organizations have not figured what aspects of these methods actually facilitate agility. Theoretically-based empirical research is needed to identify the true dimensions of software development agility.

REFERENCES

- Abrahamsson, P., Conboy, K., & Wang, X. (2009). 'Lots done, more to do': the current state of agile systems development research. *European Journal of Information Systems*, 18(4), 281–284. doi:10.1057/ejis.2009.27
- Ågerfalk, P. J., & Fitzgerald, B. (2006). Flexible and distributed software processes: Old petunias in new bowls? *Communications of the ACM*, 49(10), 26–34.
- Ambler, S. W. (2008). Has agile peaked? Scott crunches the numbers to find out. *Dr. Dobbs's Digital Digest*. Retrieved September 22, 2011, from <http://www.drdoobbs.com/architecture-and-design/207600615>
- Argyris, C. (1982). *Reasoning, learning, and action: Individual and organizational*. San Francisco, CA: Jossey-Bass.
- Armour, P. G. (2007). Agile... and offshore. *Communications of the ACM*, 50(1), 13–16. doi:10.1145/1188913.1188930
- Aydin, M. N., Harmsen, F., Stegwee, R. A., & van Slooten, K. (2005). On the adaptation of an agile information systems development method. *Journal of Database Management*, 16(4), 24–40. doi:10.4018/jdm.2005100102
- Baskerville, R., & Pries-Heje, J. (2004). Short cycle time systems development. *Information Systems Journal*, 14(3), 237–264. doi:10.1111/j.1365-2575.2004.00171.x
- Batra, D., Xia, W., VanderMeer, D., & Dutta, K. (2010). Balancing agile and structured approaches to manage large distributed software projects: A case study from the cruise line industry. *Communications of the AIS*, 27(1), 379–394.
- Beavers, P. A. (2007). Managing a large “agile” software engineering organization. In *Proceedings of the IEEE AGILE Conference* (pp. 296–303).
- Beck, K. (2000). *Extreme programming explained: Embrace change*. Reading, MA: Addison-Wesley.
- Beck, K. (2005). The XP geography: mapping your next step, a guide to planning your journey. In H. Baumeister, M. Marchesi, & M. Holcombe (Eds.), *Proceedings of the 6th International Conference on Extreme Programming and Agile Processes in Software Engineering* (LNCS 3556, pp. 287–287).
- Beck, K., & Boehm, B. (2003). Agility through discipline: A debate. *Computer*, 36(6), 44–46. doi:10.1109/MC.2003.1204374
- Blaug, M., & Rumbaugh, J. (2005). *Object-oriented modeling and design with UML* (2nd ed.). Upper Saddle River, NJ: Pearson Education.
- Blau, P. M. (1986). *Exchange and power in social life*. New Brunswick, NJ: Transaction Books.
- Boehm, B. W. (1981). *Software engineering economics*. Upper Saddle River, NJ: Prentice Hall.
- Boehm, B. W., & Turner, R. (2003). *Balancing agility and discipline: A guide for the perplexed*. Reading, MA: Addison-Wesley.
- Brechner, E. (2005). *Journey of enlightenment: the evolution of development at Microsoft*. Paper presented at the 27th International Conference on Software Engineering, St. Louis, MO.
- Brydon, M., & Vining, A. R. (2008). Adoption, improvement, and disruption: predicting the impact of open source applications in enterprise software markets. *Journal of Database Management*, 19(2), 73–94. doi:10.4018/jdm.2008040104
- Cao, L., Mohan, K., Xu, P., & Ramesh, B. (2009). A framework for adapting agile development methodologies. *European Journal of Information Systems*, 18(4), 332–343. doi:10.1057/ejis.2009.26
- Chow, T., & Cao, D. B. (2008). A survey study of critical success factors in agile software projects. *Journal of Systems and Software*, 81(6), 961–971. doi:10.1016/j.jss.2007.08.020

- Conboy, K. (2009). Agility from first principles: Reconstructing the concept of agility in information systems development. *Information Systems Research*, 20(3), 329–354. doi:10.1287/isre.1090.0236
- DeSanctis, G., & Poole, M. S. (1994). Capturing the complexity in advanced technology use: Adaptive structuration theory. *Organization Science*, 5(2), 121–147. doi:10.1287/orsc.5.2.121
- Drucker, P. F. (1997). Toward the new organization. In Hesselbein, F., Goldsmith, M., & Beckhard, R. (Eds.), *The organization of the future* (pp. 1–5). San Francisco, CA: Jossey-Bass.
- Dyba, T., & Dingsoyr, T. (2008). Empirical studies of agile software development: A systematic review. *Information and Software Technology*, 50(9–10), 833–859. doi:10.1016/j.infsof.2008.01.006
- Eckstein, J. (2004). *Agile software development in the large: Diving into the deep*. New York, NY: Dorset House.
- Eisenberg, E. M., Goodall, H. L., & Trethewey, A. (2006). *Organizational communication: Balancing creativity and constraint* (5th ed.). New York, NY: St. Martin's.
- Erickson, J., Lyytinen, K., & Siau, K. (2005). Agile modeling, agile software development, and extreme programming: the state of research. *Journal of Database Management*, 16(4), 88–100. doi:10.4018/jdm.2005100105
- Fernandez, D. J., & Fernandez, J. D. (2008). Agile project management—Agilism versus traditional approaches. *Journal of Computer Information Systems*, 49(2), 10–17.
- Fitzgerald, B., Hartnett, G., & Conboy, K. (2006). Customising agile methods to software practices at Intel Shannon. *European Journal of Information Systems*, 15(2), 200–213. doi:10.1057/palgrave.ejis.3000605
- Groise, E., & Mangin, N. (2007). Octopus: Agile software development facing our imperfect world. In *Proceedings of the IEEE AGILE Conference* (pp. 344–350).
- Gulati, R. (1995). Does familiarity breed trust? The implications of repeated ties for contractual choice in alliances. *Academy of Management Journal*, 38(1), 85–112. doi:10.2307/256729
- Heeager, L. T., & Nielsen, P. A. (2009). *Agile software development and its compatibility with a document-driven approach? A case study*. Paper presented at the Australasian Conference on Information Systems.
- Henderson-Sellers, B., & Serour, M. K. (2005). Creating a dual-agility method: The value of method engineering. *Journal of Database Management*, 16(4), 1–23. doi:10.4018/jdm.2005100101
- Highsmith, J. A. III. (1999). *Adaptive software development: A collaborative approach to managing complex systems*. New York, NY: Dorset House.
- Hilkka, M.-R., Tuure, T., & Matti, R. (2005). Is extreme programming just old wine in new bottles: a comparison of two cases. *Journal of Database Management*, 16(4), 41–61. doi:10.4018/jdm.2005100103
- Hofstede, G., & Hofstede, G. J. (2005). *Cultures and organizations: Software for the mind*. New York, NY: McGraw-Hill.
- Holland, J. H. (1992). Complex adaptive systems. *Daedalus*, 121(1), 17–30.
- Jacques, E. (1996). In praise of hierarchy. In Shafritz, J. M., & Ott, J. S. (Eds.), *Classics of organizational theory* (pp. 245–253). Orlando, FL: Harcourt Brace.
- Jain, N. (2006). Offshore agile maintenance. In *Proceedings of the IEEE AGILE Conference*, Minneapolis, MN.
- Jensen, M. C., & Meckling, W. H. (1976). Theory of the firm: Managerial behavior, agency costs and ownership structure. *Journal of Financial Economics*, 3(4), 305–360. doi:10.1016/0304-405X(76)90026-X
- Kahkonen, T. (2004). Agile methods for large organisations - Building communities of practice. In *Proceedings of the AGILE Development Conference*, Salt Lake City, UT.
- Kelley, J. E. (1961). Critical-path planning and scheduling: Mathematical basis. *Operations Research*, 9(3), 296–320. doi:10.1287/opre.9.3.296
- Korkala, M., & Abrahamsson, P. (2007). Communication in distributed agile development: A case study. In *Proceedings of the 33rd EUROMICRO Conference on Software Engineering and Advanced Applications* (pp. 203–210).
- Larman, C. (2003). *Agile and iterative development: A manager's guide*. Reading, MA: Addison-Wesley.
- Layman, L., Williams, L., Damian, D., & Bures, H. (2006). Essential communication practices for Extreme Programming in a global software development team. *Information and Software Technology*, 48(9), 781–794. doi:10.1016/j.infsof.2006.01.004

- Lee, G., & Xia, W. (2010). Toward agile: An integrated analysis of quantitative and qualitative field data. *Management Information Systems Quarterly*, 34(1), 87–114.
- Lin, Z., Xu, B., & Xu, Y. (2011). A study of open source software development from control perspective. *Journal of Database Management*, 22(1), 26–42. doi:10.4018/jdm.2011010102
- Lindvall, M., Basili, V., Boehm, B., Costa, P., Dangle, K., Shull, F., et al. (2002). *Empirical findings in agile methods*. Paper presented at the Second XP Universe and First Agile Universe Conference, Chicago, IL.
- Lindvall, M., Muthig, D., Dagnino, A., Wallin, C., Stupperich, M., & Kiefer, D. (2004). Agile software development in large organizations. *Computer*, 37(12), 26–34. doi:10.1109/MC.2004.231
- Lyytinen, K., & Rose, G. M. (2006). Information system development agility as organizational learning. *European Journal of Information Systems*, 15, 181–199. doi:10.1057/palgrave.ejis.3000604
- Malcolm, D. G., Roseboom, J. H., Clark, C. E., & Fazar, W. (1959). Application of a technique for research and development program evaluation. *Operations Research*, 7(5), 646–669. doi:10.1287/opre.7.5.646
- Maruping, L. M., Venkatesh, V., & Agarwal, R. (2009). A control theory perspective on agile methodology use and changing user requirements. *Information Systems Research*, 20(3), 377–399. doi:10.1287/isre.1090.0238
- McAvoy, J., & Butler, T. (2009). The role of project management in ineffective decision making within Agile software development projects. *European Journal of Information Systems*, 18(4), 372–383. doi:10.1057/ejis.2009.22
- Meredith, S., & Francis, D. (2000). Journey towards agility: The agile wheel explored. *The TQM Magazine*, 12(2), 137–143. doi:10.1108/09544780010318398
- Meso, P., & Jain, R. (2006). Agile software development: Adaptive system principles and best practices. *Information Systems Management*, 23(5), 19–29. doi:10.1201/1078.10580530/46108.23.3.20060601/93704.3
- Misra, S. C., Kumar, V., & Kumar, U. (2009). Identifying some important success factors in adopting agile software development practices. *Journal of Systems and Software*, 82, 1869–1890. doi:10.1016/j.jss.2009.05.052
- Nerur, S., & Balijepally, V. G. (2007). Theoretical reflections on agile development methodologies. *Communications of the ACM*, 50(3), 79–83. doi:10.1145/1226736.1226739
- Nerur, S., Mahapatra, R., & Mangalaraj, G. (2005). Challenges of migrating to agile methodologies. *Communications of the ACM*, 48(5), 72–78. doi:10.1145/1060710.1060712
- Nord, R. L., & Tomayko, J. E. (2006). Software architecture-centric methods and agile development. *IEEE Software*, 23(2), 47–53. doi:10.1109/MS.2006.54
- Nuseibeh, B., & Easterbrook, S. (2000). *Requirements engineering: A roadmap*. Paper presented at the International Conference on Software Engineering, Limerick, Ireland.
- O'Connor, D. E., & Faille, C. C. (2000). *Basic economic principles: A guide for students*. Westport, CT: Greenwood Press.
- Ouchi, W. G. (1979). A conceptual framework for the design of organizational control mechanisms. *Management Science*, 25(9), 833–848. doi:10.1287/mnsc.25.9.833
- Poppendieck, M., & Poppendieck, T. (2003). *Lean software development: An agile toolkit*. Reading, MA: Addison-Wesley.
- Procaccino, J. D., Verner, J. M., Overmyer, S. P., & Darter, M. E. (2002). Case study: Factors for early prediction of software development success. *Information and Software Technology*, 44(1), 53–62. doi:10.1016/S0950-5849(01)00217-8
- Project Management Institute. (2004). *A guide to the project management body of knowledge*. Newtown Square, PA: Project Management Institute.
- Qumer, A., & Henderson-Sellers, B. (2008). An evaluation of the degree of agility in six agile methods and its applicability for method engineering. *Information and Software Technology*, 50(4), 280–295. doi:10.1016/j.infsof.2007.02.002
- Ramesh, B., Cao, L., Mohan, K., & Xu, P. (2006). Can distributed software development be agile? *Communications of the ACM*, 49(10), 41–46. doi:10.1145/1164394.1164418
- Reeves, W. W. (1999). *Learner-centered design: A cognitive view of managing complexity in product, information, and environmental design*. Thousand Oaks, CA: Sage.

- Sarker, S., & Sarker, S. (2009). Exploring agility in distributed information systems development teams: an interpretive study in an offshoring context. *Information Systems Research*, 20(3), 440–461. doi:10.1287/isre.1090.0241
- Schermerhorn, J. R., Hunt, J. G., & Osborn, R. N. (2003). *Organizational behaviour*. New York, NY: John Wiley & Sons.
- Schwaber, K. (2005). Agile project management. In H. Baumeister, M. Marchesi, & H. Holombe (Eds.), *Proceedings of the 6th International Conference on Extreme Programming and Agile Processes in Software Engineering* (LNCS 3556, pp. 277–277).
- Schwaber, K., & Beedle, M. (2002). *Agile software development with Scrum*. Upper Saddle River, NJ: Prentice Hall.
- Siau, K., Long, Y., & Ling, M. (2010). Toward a unified model of information systems development success. *Journal of Database Management*, 21(1), 80–101. doi:10.4018/jdm.2010112304
- Simon, H. A. (1957). *Models of man: Social and rational; Mathematical essays on rational human behavior in a social setting*. New York, NY: John Wiley & Sons.
- Stevens, W. P., Myers, G. J., & Constantine, L. L. (1999). Structural design. *IBM Systems Journal*, 38(2-3), 231–256. doi:10.1147/sj.382.0231
- Turk, D., France, R., & Rumpe, B. (2005). Assumptions underlying agile software-development processes. *Journal of Database Management*, 16(4), 62–87. doi:10.4018/jdm.2005100104
- Vidgen, R., & Wang, X. (2009). Coevolving systems and the organization of agile software development. *Information Systems Research*, 20(3), 355–376. doi:10.1287/isre.1090.0237
- Vinekar, V., Slinkman, C. W., & Nerur, S. (2006). Can Agile and traditional systems development approaches coexist? An ambidextrous view. *Information Systems Management*, 25(3), 31–42. doi:10.1201/1078.10580530/46108.23.3.20060601/93705.4
- Vroom, V. H. (1964). *Work and motivation*. New York, NY: John Wiley & Sons.
- Williamson, O. E. (1979). Transaction-cost economics: The governance of contractual relations. *The Journal of Law & Economics*, 22(2), 233–261. doi:10.1086/466942

Dinesh Batra is a professor in the decision sciences and information systems in the College of Business Administration at the Florida International University. His publications have appeared in Management Science, Journal of MIS, Communications of the ACM, Communications of the Association for Information Systems, European Journal of Information Systems, International Journal of Human Computer Studies, Journal of Database Management, Computers and Operations Research, Data Base, Information and Management, Decision Support Systems, Requirements Engineering Journal, Information Systems Management, and others. He is a co-author of the book Object-Oriented Systems Analysis and Design published by Pearson Prentice-Hall. He has served as the President of the AIS SIG on Systems Analysis and Design (SIGSAND).

Debra VanderMeer is an assistant professor in the college of business at Florida International University. Her research interests focus on applying concepts developed in computer science and information systems to solve real-world problems. She is widely published in well-known journals, such as Management Science, ACM Transactions on Database Systems, and IEEE Transactions on Knowledge and Data Engineering, as well as prestigious conference proceedings, including the International Conference on Data Engineering, International Conference on Distributed Computing Systems, and the Very Large Database Conference. She also has significant professional experience in the software industry. She has served in software engineering and managerial roles in large companies, as well as early-stage venture-funded software enterprises. She received her doctorate from the Georgia Institute of Technology.

Kaushik Dutta is an associate professor of information systems in the school of computing at the National University of Singapore. His research interest is enterprise IT Infrastructure and software development. He has published articles in journals such as Management Science, Journal of Computing, and ACM Transactions on Database Systems. Dr. Dutta also has several publications in various IEEE and ACM conference proceedings. He is the area editor (Database and Data Management) for Elsevier Journal of Systems and Software. Previously Dr. Dutta served as an Associate Professor at Florida International University (FIU). Prior to joining FIU, Dr. Dutta was Director of Engineering for Chutney Technologies, funded by KPCB Venture Capital, that developed solutions to improve the scalability and performance of enterprise Web applications. He received his doctorate in Information Systems from the Georgia Institute of Technology.